Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

VHDL-Übung 700: Endliche Automaten: Snackautomat

M.Sc. Eike Trumann, M.Sc. Jasper Homann

Prof. Dr.-Ing. Guillermo Payá Vayá

06. Dezember 2024

Aufgabe VHDL-700: *Snackautomat*

Im folgenden ist ein System gegeben, das durch eine Hardwareschaltung gesteuert werden soll.

Ein Automat soll verschiedene Snacks bereitstellen. Die Ausgabe soll basierend auf einer FSM-Steuerung erfolgen. Die Steuerung hat folgende Eingänge (Buttons):

VHDL-700

Funktion	Signal
Snack-Auswahl	selection
Abbruchtaste	cancel
Ausreichende Zahlung (externer Kassenautomat)	payment_ok

Die Ausgabe des Automaten wird durch folgende Signale ausgegeben:

Funktion	Signal
Auswurf des ausgewählten Snacks (Trigger)	eject
Abbruch (an externen Kassenautomaten)	paycancel
Preis des ausgewählten Snacks (an externen Kassenautomaten)	cost

- Der Automat soll standardmäßig (im IDLE Zustand) die **Leer**-Selection haben und einen Preis von 0€ anzeigen.
- **Nach** getätigter Eingabe (*selection*), wird der Preis der aktuellen Auswahl an den Kassenautomaten ausgegeben (*cost*). Es können verschiedene Snacks ausgewählt werden. Das Signal zur Wahl eines Snacks (*selection*) wird durch Drucktaster realisiert. Es soll

Schoko, Cookie, sowie **keine Betätigung** darstellen können.
Zu beachten: Ein Wechsel des ausgewählten Snacks ist (zur Vereinfachung) nicht vorgesehen.

- **Nach** der Zahlung des aktuellen Preises (durch den externen Kassenautomaten) wird das Signal (*ok*) aktiviert und der gewählte Snack wird ausgegeben/der Auswurf getriggert (*eject*).
- **Nach** dem Auswurf (*eject*) wird der Automat in den IDLE-Zustand zurückgesetzt. Dies erfolgt ebenso durch die Abbruchtaste (*cancel*). Der Abbruch über die Abbruchtaste wird an den Kassenautomaten weitergegeben (*paycancel*) und keine Ausgabe (*eject*) ausgelöst. Der Abbruch tritt in jedem Fall ein (auch wenn zeitgleich gezahlt wird –> das Geld gibt die Kasse zurück).

Das Signal für den Preis (*cost*) soll eine Präzision von 0,25 € haben und einen Maximalpreis von 4 € darstellen können.

Snack	`select_i`	Preis	Binäre Codierung des Preises
keine Auswahl	`00`	0	00000
Schoko	`01`	0,75 €	00011
Cookie	`10`	2,50 €	01010
Beides gleichzeitig gedrückt	`11`	0	00000

Es ist hilfreich, zunächst eine Zeichnung der FSM als Moore-Automat anzufertigen und diese dann in VHDL umzusetzen. Das technische Signal `areset_ni` ist ein bei 0 aktiver Reset, der nicht durch den Kunden bedient werden kann und der die FSM immer sofort (asynchron) in den Startzustand zurücksetzt. Dieses dient ausschließlich Testzwecken, dieser Eingang muss nicht in der Zeichnung berücksichtigt werden.

Für unsinnige Eingaben soll folgendes gelten:

- Werden im Grundzustand die Select-Taste und Cancel gleichzeitig gedrückt, bleibt die FSM im Grundzustand (IDLE).
- Gibt die Bezahleinheit ein `payment_ok` im Grundzustand, bleibt die FSM im Grundzustand.
- Wird während der Ausgabe Cancel gedrückt, hat dieses keine weiteren Auswirkungen, die Ausgabe wird abgeschlossen und die FSM kehrt in ihren Grundzustand zurück.
- Wird während des Abbruchs nach Drücken der Cancel-Taste eine vollständige Bezahlung gemeldet, wird `paycancel` ausgegeben und die Bezahleinheit gibt das Geld zurück.
- Werden beide Snacks gleichzeitig ausgewählt, verbleibt die FSM im Grundzustand.

Es soll das folgende Template für die Implementierung der gegebenen FSM in VHDL verwendet werden. Sequentielle und kombinatorische Logik sollen in zwei getrennten Prozesse platziert werden.

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;
entity snackautomat_fsm is
    port(
        clk_i, areset_ni : in  std_ulogic;
        cancel_i         : in  std_ulogic;
        payment_ok_i     : in  std_ulogic;
        -- Zwei Auswahlmoeglichkeiten in One-Hot-Codierung
        selection_i      : in  std_ulogic_vector(1 downto 0);
        eject_o          : out std_ulogic;
        paycancel_o      : out std_ulogic;
        cost_o           : out std_ulogic_vector(4 downto 0)
    );
end entity snackautomat_fsm;

architecture rtl of snackautomat_fsm is
  type fsm_t is (IDLE, ...); -- add more states
  signal fsm, fsm_nxt : fsm_t;
begin
  -- your code here
end architecture rtl;
```